SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
COMPUTER SCIENCE AND ENGINEERING

CSA47 – DEEP LEARNING

ASSIGNMENT 2 | CO2: Deep Feed-Forward and Convolutional Network Design

NAME: NAVEEN V
REG:192424126

Task 1 – Deep Feed-Forward Network Architecture (5 Marks)

a) Architecture. The inputs are rainfall, temperature, soil moisture and fertilizer quantity, so the network is 4 → 8 → 4 → 1:

Input layer: 4 neurons | Hidden layer 1: 8 neurons | Hidden layer 2: 4 neurons | Output layer: 1 neuron (crop yield)

The output layer has one neuron with a linear activation because yield is a continuous value (regression). The hidden layers use ReLU.

b) Dimensions. For a layer with n_{in} inputs and n_{out} neurons, W has shape $n_{out} \times n_{in}$ and b has shape $n_{out} \times 1$.

Layer	Weight matrix	Bias vector	Parameters
Input → Hidden 1	$W^{(1)}$: 8 × 4	$b^{(1)}$: 8 × 1	32 + 8 = 40
Hidden 1 → Hidden 2	$W^{(2)}$: 4 × 8	$b^{(2)}$: 4 × 1	32 + 4 = 36
Hidden 2 → Output	$W^{(3)}$: 1 × 4	$b^{(3)}$: 1 × 1	4 + 1 = 5
Total			**81**

c) Forward propagation. The input vector x (4 × 1) is normalized and passed layer by layer:

$$a^{(1)} = ReLU(W^{(1)}x + b^{(1)}) \quad a^{(2)} = ReLU(W^{(2)}a^{(1)} + b^{(2)}) \quad \hat{y} = W^{(3)}a^{(2)} + b^{(3)}$$

Each neuron takes a weighted sum of the previous layer's outputs, adds its bias and applies the activation. The 8 values from the first hidden layer become the inputs of the second hidden layer (4 values), which feeds the single output neuron. That output is the predicted crop yield.

Task 2 – Forward Propagation and Activation Functions (2 Marks)

a) Inputs 0.6, 0.8, 0.5; weights 0.4, 0.3, 0.2; bias 0.1.

$$z = (0.6)(0.4) + (0.8)(0.3) + (0.5)(0.2) + 0.1 = 0.24 + 0.24 + 0.10 + 0.10 = \mathbf{0.68}$$

b) ReLU(z) = max(0, z) = max(0, 0.68) = **0.68**. Since the weighted sum is positive the neuron is active and passes the value on unchanged. If z had been negative the output would be 0 and the neuron would stay silent for that input.

Task 3 – Gradient-Based Learning and Loss Calculation (6 Marks)

a) A loss function gives a single number for how far the predictions are from the actual values. Without it there is nothing to minimize and no direction in which to adjust the weights. For regression, Mean Squared Error (squared error per sample) is suitable because yield is continuous, it is smooth and differentiable, and it penalizes large errors more heavily.

b) $y = 50$ and $\hat{y} = 42$:

$$L = (y - \hat{y})^2 = (50 - 42)^2 = 8^2 = \mathbf{64}$$

c) The loss of 64 shows the network under-predicts the yield by 8 units, which is a sizeable error. A loss of 0 would mean a perfect prediction, and the training goal is to push this value down.

d) Gradient-based learning computes the gradient of the loss with respect to every weight and bias (using backpropagation). The gradient tells how the loss changes when a parameter changes. Each parameter is then moved a small step against its gradient, $\theta \leftarrow \theta - \eta(\partial L/\partial \theta)$, and this is repeated over many iterations until the loss stops improving.

Task 4 – Gradient Descent and Weight Update (4 Marks)

a) $w_{new} = w - \eta(\partial L/\partial w) = 0.5 - (0.1)(0.8) = 0.5 - 0.08 = \mathbf{0.42}$

b) The gradient points in the direction in which the loss increases fastest. To reduce the loss we have to move the opposite way, so the update subtracts the gradient. Here the gradient is positive, so decreasing the weight lowers the loss.

c) A very small learning rate makes training extremely slow and it can get stuck on plateaus or poor local regions. A very large learning rate overshoots the minimum, makes the loss oscillate or even increase, and can make training diverge. A moderate value, often with a decay schedule, works best.

Task 5 – Back-Propagation Through Multiple Layers (3 Marks)

a) After the forward pass the loss is computed at the output. Backpropagation then moves backwards from the output layer through hidden layer 2 and hidden layer 1 to the input, calculating how much each weight contributed to the error. Weights that influenced the output more get larger gradients.

b) The network is a chain of composed functions, so the chain rule breaks the gradient into a product of local derivatives. For a first-layer weight: $\partial L/\partial W^{(1)} = \partial L/\partial \hat{y} \cdot \partial \hat{y}/\partial a^{(2)} \cdot \partial a^{(2)}/\partial a^{(1)} \cdot \partial a^{(1)}/\partial W^{(1)}$. The gradient from the later layer is reused for the earlier one, so it is calculated efficiently.

c) Once every gradient is known, each weight and bias is updated with $W \leftarrow W - \eta \partial L/\partial W$. The next forward pass then gives a lower error, and the cycle repeats.

Task 6 – Activation Functions and Vanishing/Exploding Gradients (3 Marks)

a) During backpropagation the gradient is multiplied by the activation derivative at every layer. Sigmoid has a maximum derivative of only 0.25 and tanh saturates near ±1, so repeated multiplication of small numbers shrinks the gradient exponentially. The early layers then receive almost no signal and learn very slowly. In the opposite case, large weights with derivatives above 1 make the gradients explode.

b) Comparison for hidden layers:

Function	Range / derivative	Advantages	Drawbacks
Sigmoid	(0, 1); max slope 0.25	Smooth, probability-like output	Saturates, vanishing gradient, not zero-centred
tanh	(−1, 1); max slope 1	Zero-centred, stronger gradient than sigmoid	Still saturates for large |x|
ReLU	[0, ∞); slope 1 for x > 0	Cheap, no saturation for positive values, fast training	Dying ReLU (neurons stuck at 0)

c) ReLU (or Leaky ReLU) with proper weight initialization such as He initialization, batch normalization and gradient clipping keeps the gradients healthy, so deep networks train faster and more stably. Sigmoid and tanh in deep hidden layers make training slow, so sigmoid is kept only for binary output layers.

Task 7 – Back-Propagation and Optimization Strategy (4 Marks)

a) The learning rate sets the step size. If it is too small the loss falls slowly; if it is too large the updates overshoot, so the loss fluctuates or diverges.

b) In one iteration a mini-batch goes through the forward pass to get predictions, the loss is computed, backpropagation calculates the gradients of all parameters, and gradient descent uses those gradients to update the weights. Backpropagation finds the gradients; gradient descent applies them.

c) *Experiment 1 (slow decrease):* increase the learning rate gradually, use momentum or Adam, normalize the inputs and add batch normalization. *Experiment 2 (fast but fluctuating):* reduce the learning rate, use a decay schedule, increase the batch size and use gradient clipping.

d) Plotting training and validation loss shows whether the model is converging, oscillating, underfitting or overfitting. It also shows when to stop early or change the learning rate, which saves training time.

Task 8 – Regularization and Overfitting Control (4 Marks)

a) The network is **overfitting**. The training accuracy is 99% but validation accuracy is only 78%, a gap of 21 points, which means it has memorized the training data instead of learning patterns that generalize.

b) Regularization adds a penalty on the weights to the loss. **L1** adds $\lambda\Sigma|w|$, which pushes many weights to exactly zero and gives a sparse model with feature selection. **L2** adds $\lambda\Sigma w^2$, which shrinks all weights smoothly and keeps them small. Both limit the effective complexity of the

model.

c) $L_{reg} = (1/N)\,\Sigma(y_i - \hat{y}_i)^2 + \lambda\Sigma w^2$

Here λ is the regularization parameter. A value of 0 means no regularization, a small λ gives a weak penalty, and a very large λ forces the weights towards zero and causes underfitting. It is chosen using validation data.

d) Dropout randomly switches off a fraction of neurons (e.g. 20–50%) in each training step. Neurons cannot depend on specific partners, so the network learns more robust features and behaves like an ensemble of many smaller networks. All neurons are used at test time.

Task 9 – Convolution Operation and Feature Extraction (3 Marks)

a) Leaf images have local patterns such as spots, edges and texture. Convolutional layers use small filters that slide over the image, so they detect these patterns anywhere (weight sharing), keep spatial structure and need far fewer parameters than a fully connected layer.

b) **Example.** A 4 × 4 image convolved with a 3 × 3 vertical-edge filter, stride 1, no padding, gives a 2 × 2 output.

Image **Filter** **Output**

1	2	0	1
0	1	3	1
2	1	0	0
1	0	1	2

1	0	−1
1	0	−1
1	0	−1

0	2
−1	−1

Top-right position (rows 1–3, columns 2–4): (2+1+1) − (1+1+0) = 4 − 2 = ***2****. The other values are computed in the same way.*

Interpretation: a large positive value (2) means a strong vertical brightness change (an edge) at that position, 0 means no change, and negative values show an edge in the opposite direction. On a real leaf the filter would highlight lesion boundaries.

c) **Filters** are small weight matrices that learn to detect features such as edges or spots. **Stride** is the number of pixels the filter moves each step; a larger stride gives a smaller feature map. **Padding** adds a border of zeros so edge pixels are used properly and the output size can be preserved. Output size = (W − F + 2P)/S + 1.

Task 10 – CNN Architecture and Pooling (4 Marks)

a) **Input image:** raw pixels (e.g. 64 × 64 × 3). **Convolution:** filters extract local features. **ReLU:** adds non-linearity and keeps only positive responses. **Pooling:** down-samples feature maps. **Second Conv + ReLU + Pooling:** combine the first features into higher-level ones. **Fully connected layer:** combines all features for the decision. **Output:** softmax or sigmoid giving healthy / diseased probabilities.

b) Pooling takes a summary value from each small window. Max pooling with a 2 × 2 window and stride 2 keeps the maximum of every 2 × 2 block, so a 64 × 64 map becomes 32 × 32 (height and width halved, 75% fewer values). This cuts computation and parameters, gives some tolerance to

small shifts and reduces overfitting.

c) A larger filter sees a wider area but costs more parameters and shrinks the map more. A larger stride shrinks the map faster and may skip fine detail. More filters create more feature maps (more depth) and capture more patterns, but increase memory and computation.

> *Example: 64 × 64 × 3 input → Conv 3 × 3, 16 filters, padding 1 → 64 × 64 × 16 → Pool → 32 × 32 × 16 → Conv, 32 filters → 32 × 32 × 32 → Pool → 16 × 16 × 32 (8192 values flattened into the fully connected layer).*

d) The first layers learn simple features such as edges and colour patches. Deeper layers combine them into textures and spot patterns, and the last layers recognize complete disease signs. Every layer builds on the one before, giving a hierarchy from simple to complex.

Task 11 – Comparative Evaluation of Feed-Forward and CNN Models (4 Marks)

a) Rainfall, temperature, soil moisture and fertilizer quantity are a small structured vector of independent numerical values with no spatial layout, so a feed-forward network can learn their combined non-linear effect on yield directly and cheaply.

b) Leaf images have a spatial structure and nearby pixels are related. A CNN uses local filters and shared weights to detect spots and texture wherever they appear, which suits images.

c) Comparison:

Aspect	Model A (Feed-forward)	Model B (CNN)
Input representation	Vector of 4 numerical features	Image tensor (H × W × 3)
Feature extraction	Hand-chosen features; learned combinations	Automatic, hierarchical learning of edges, texture, shapes
Parameter organization	Dense weight matrices (every input to every neuron)	Small shared filters plus a dense classifier
Computation	Very low; CPU is enough	High; GPU recommended

d) **Recommendation:** use Model A (feed-forward) for crop-yield prediction and Model B (CNN) for leaf-disease classification. Each architecture matches its data type: a CNN on four numbers would be wasteful, and a feed-forward network on raw images would need huge numbers of parameters and ignore spatial structure.

Task 12 – Design Challenge: Optimize and Redesign the Deep Learning System (8 Marks)

A. Impact Analysis (2 marks)

a) Moving from 10,000 to 500,000 numerical records increases data loading, memory and training time, so mini-batch training, efficient data pipelines and possibly GPUs are needed. The 100,000 leaf images add a new data type with much larger input size, which needs a convolutional architecture, far more compute and memory, and image preprocessing.

b) A feed-forward network flattens an image and loses spatial relationships. It needs a huge number of weights (a 224 × 224 × 3 image has 150,528 inputs), is not translation tolerant, overfits easily and is computationally expensive.

B. Evaluation of the Existing Design (2 marks)

a) High training accuracy with lower validation performance means the existing model overfits and does not generalize well, and the long training time shows it is inefficient at the new scale.

b) No. Adding more layers and neurons increases capacity, which usually worsens overfitting, and adds to computation and training time. Better data handling, regularization, better optimization and a suitable architecture are more effective.

C. Adaptation or Redesign (2 marks)

a) **Numerical data:** feed-forward 4 → 64 → 32 → 1 with ReLU, batch normalization and dropout (0.2), trained on 500,000 records with MSE loss. **Leaf images:** a CNN with convolution blocks (Conv → BatchNorm → ReLU → MaxPool) and filters increasing 32 → 64 → 128, followed by global average pooling, a dense layer with dropout and a softmax output; transfer learning is used since 100,000 images are available.

b) Use Adam (learning rate 0.001) or SGD with momentum, a learning-rate schedule, L2 weight decay (e.g. 1e-4), dropout, batch normalization, early stopping and ReLU (Leaky ReLU if neurons die) in hidden layers; linear output for regression and softmax/sigmoid for classification.

D. Evidence-Based Defence (2 marks)

a) Split the data into training / validation / test sets (70/15/15) and use the same split for both systems, with fixed random seeds and 3 repeated runs. Record training and validation loss, time per epoch, total training time and memory. Regression metrics: MAE, RMSE and R^2. Classification metrics: accuracy, precision, recall, F1-score, ROC-AUC and the confusion matrix. Finally compare the validation curves for overfitting.

b) The existing system should be **modified and extended**: the feed-forward network is kept for numerical yield prediction but with regularization, mini-batching and a better optimizer, and a new CNN is added for images, because a feed-forward network cannot handle image data well. The decision is confirmed if the redesigned system shows a smaller train–validation gap, better validation metrics and shorter training time.

Additional Design Considerations

Optimizer comparison:

Optimizer	Idea	Remarks
SGD	Updates after every single sample	Noisy and slow per epoch; poor use of hardware
Mini-batch SGD	Updates after a small batch (e.g. 32–256)	Standard choice; stable and efficient
Momentum	Adds a moving average of past gradients	Faster, reduces oscillation
RMSProp	Divides step by running average of squared gradients	Adaptive rates; good for non-stationary problems
Adam	Momentum + RMSProp with bias correction	Fast convergence, good default; can generalize slightly worse than SGD

Learning-rate schedule: use a warm-up for the first 5 epochs, starting at 1e-4 and rising to 1e-3 with Adam, then cosine decay to 1e-5. Reduce the rate by a factor of 10 on plateau as an alternative.

When fine-tuning pretrained CNN layers, use a lower rate (about 1e-4).

Label-preserving image augmentation: horizontal and vertical flips, rotation up to ±30°, small zoom and crop, small brightness and contrast changes, and light Gaussian blur or noise to mimic camera variation. Avoid strong colour or hue changes and large crops that remove the lesion, because they can alter the disease label. Augmentation is applied only to training images.

CNN comparison:

Model	Parameters (approx.)	Typical input	Remarks
LeNet-5	~60 thousand	32 × 32 grey	Too small for detailed leaf texture
AlexNet	~60 million	227 × 227	Large dense layers, heavy and prone to overfitting
ResNet-50	~25 million	224 × 224	Skip connections ease deep training; strong accuracy
EfficientNet-B0	~5 million	224 × 224	Best accuracy per computation; efficient

For 224 × 224 leaf images and a limited computation budget, I would fine-tune a pretrained **EfficientNet-B0** (with ResNet-50 as a strong baseline), since it gives high accuracy with few parameters.

Conclusion

A feed-forward network suits the numerical crop data, while a CNN is the right choice for leaf images. Using proper loss functions, backpropagation with gradient descent, suitable activations, regularization and good optimization, the redesigned system gives better generalization at a larger data scale. The final design should be confirmed with the evaluation experiment described above.